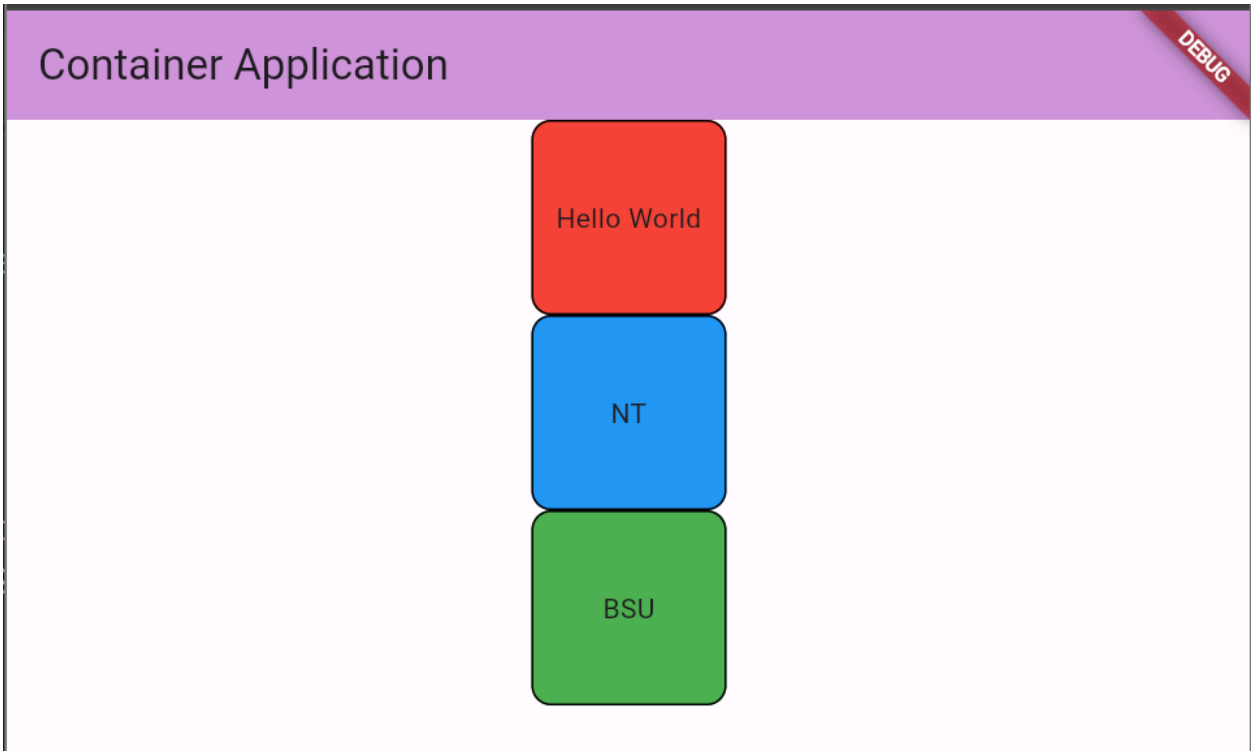
Container Widgets



The **Container widget** in Flutter is a versatile widget used for creating a rectangular visual element. It can be customized with various properties to control its size, padding, margins, alignment, decoration, and more. It's commonly used as a building block for layout design.

Container Widget Overview

- Purpose: To create a rectangular visual element with various customizable properties.
- Common Use Cases:
    - Wrapping other widgets to add padding, margins, borders, or background colors.
    - Creating basic layouts and positioning elements within the app.

Basic Structure

Here is a basic example of a Container widget:

```
Container(
  child: Text('Hello, World!'),
)
```

Properties

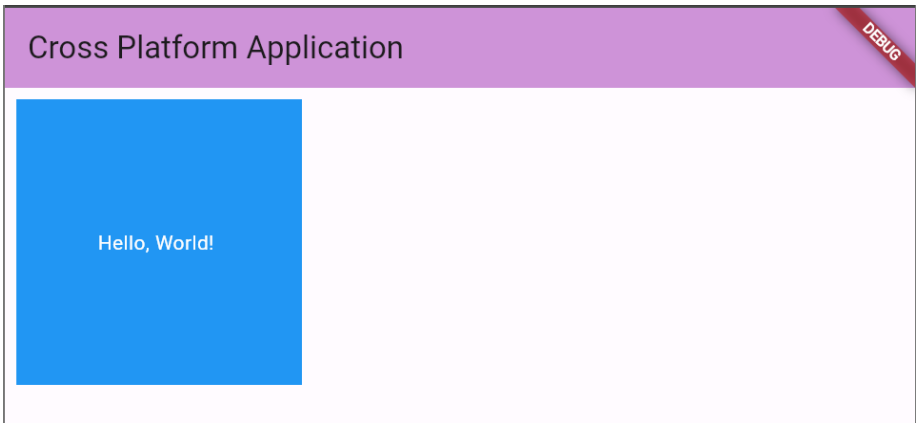
The Container widget has several properties that allow for extensive customization:

1. **alignment**:
    - **Type:** AlignmentGeometry
    - **Description:** Aligns the child within the container. Common values include Alignment.center, Alignment.topLeft, etc.
2. **padding**:
    - **Type:** EdgeInsetsGeometry
    - **Description:** Adds padding inside the container.
3. **margin**:
    - **Type:** EdgeInsetsGeometry
    - **Description:** Adds space outside the container.
4. **color:**
    - **Type:** Color
    - **Description:** Sets the background color of the container.
5. **decoration**:
    - **Type**: Decoration
    - **Description**: Provides more advanced decoration options, such as gradients, borders, and shapes.
6. **foregroundDecoration**:
    - **Type**: Decoration
    - **Description**: Similar to decoration, but appears in front of the child.
7. **width**:
    - **Type**: double
    - **Description**: Sets the width of the container.
8. **height**:
    - **Type**: double
    - **Description**: Sets the height of the container.

9. **constraints**:
    - **Type**: BoxConstraints
    - **Description**: Adds constraints to the container's size.
10. **transform**:
    - **Type**: Matrix4
    - **Description**: Applies a transformation (e.g., rotation, scaling) to the container.
11. **child**:
    - **Type**: Widget
    - **Description**: The widget inside the container.
12. **clipBehavior**:
    - **Type**: Clip
    - **Description**: Controls how the content should be clipped if it overflows the container.

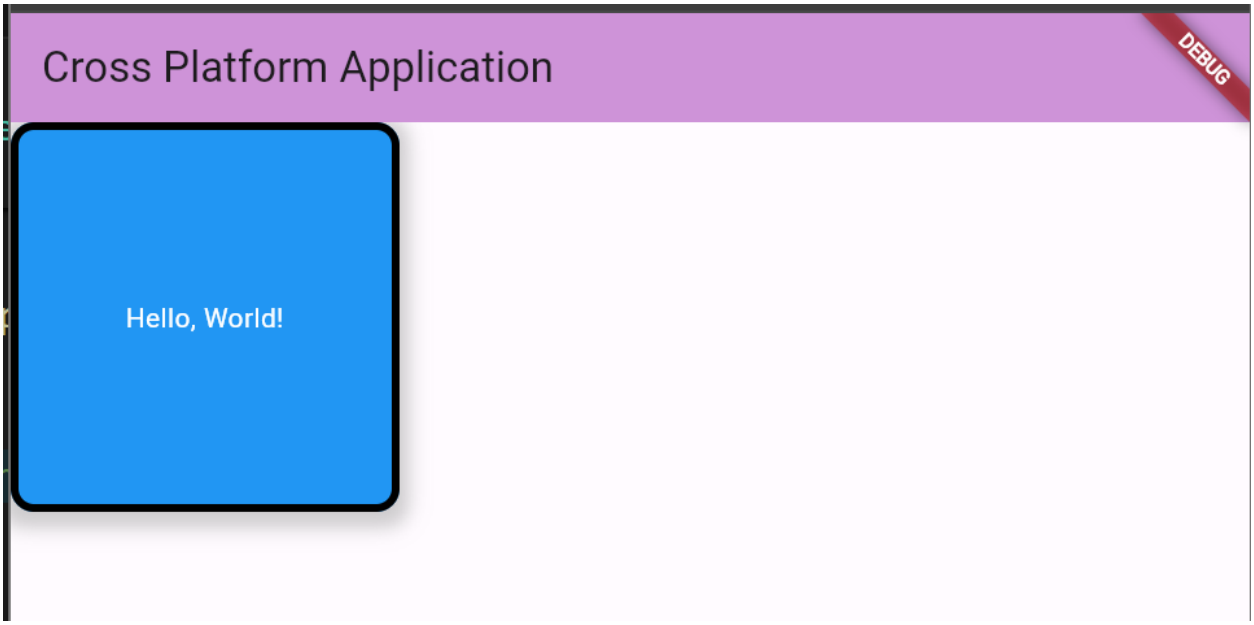
Example with Properties

```dart
Container(
  alignment: Alignment.center,
  padding: EdgeInsets.all(16.0),
  margin: EdgeInsets.all(8.0),
  color: Colors.blue,
  width: 200.0,
  height: 200.0,
  child: Text(
    'Hello, World!',
    style: TextStyle(color: Colors.white),
  ),
)
```



Example with Decoration

```dart
Container(
  width: 200.0,
  height: 200.0,
  decoration: BoxDecoration(
    color: Colors.blue,
    border: Border.all(color: Colors.black, width: 4.0),
    borderRadius: BorderRadius.circular(12.0),
    boxShadow: [
      BoxShadow(
        color: Colors.grey.withOpacity(0.5),
        spreadRadius: 5,
        blurRadius: 7,
        offset: Offset(0, 3), // changes position of shadow
      ),
    ],
  ),
  child: Center(
    child: Text(
      'Hello, World!',
      style: TextStyle(color: Colors.white),
    ),
  ),
)
```



Box Decoration Class

The BoxDecoration widget in Flutter is used to decorate a Container widget. It allows you to specify various properties like color, border, shape, gradient, and more. This is particularly useful when you want to style the appearance of your containers.

BoxDecoration Overview

- **Purpose**: To decorate a Container or other boxes with properties like color, border, gradient, shadows, and more.
- **Common Use Cases**:
    - Styling containers with backgrounds, borders, and rounded corners.
    - Adding shadows and gradients to containers.
    - Creating complex and visually appealing UI components.

Basic Structure

Here is a basic example of a Container with BoxDecoration:

```
Container(
  decoration: BoxDecoration(
    color: Colors.blue,
  ),
  child: Text('Hello, World!'),
)
```

Properties

The **BoxDecoration** class has several properties that allow for extensive customization:

1. **color**:
    - Type: Color
    - Description: Sets the background color of the container.
2. **border**:
    - Type: Border
    - Description: Adds a border around the container. You can customize the color, width, and style of the border.
3. **borderRadius**:
    - Type: BorderRadius
    - Description: Rounds the corners of the container. Common values include BorderRadius.circular(12.0).
4. **boxShadow**:
    - Type: List<BoxShadow>
    - Description: Adds shadow effects around the container.
5. **gradient**:
    - Type: Gradient
    - Description: Fills the container with a gradient. Common gradients include LinearGradient and RadialGradient.
6. **image**:
    - Type: DecorationImage

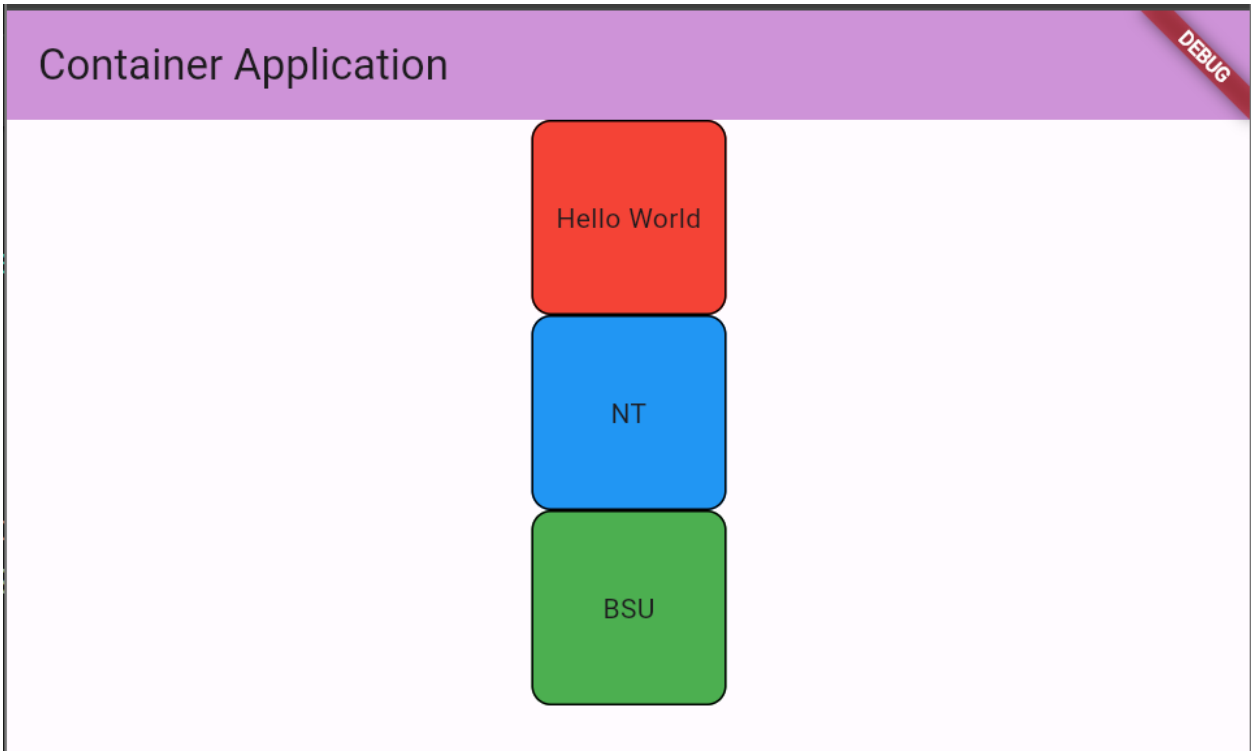
- Description: Adds a background image to the container.

7. **shape**:
    - Type: BoxShape
    - Description: Sets the shape of the container. Common values are BoxShape.rectangle and BoxShape.circle.
8. **backgroundBlendMode**:
    - Type: BlendMode
    - Description: Blends the background color and image.

Full code of sample UI above

```dart
class MyHomePage extends StatelessWidget {
  const MyHomePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Container Application'),
        backgroundColor: Colors.purple[200],
      ),
      body: Container(
        height: 100,
        width: 100,
        margin: const EdgeInsets.fromLTRB(10, 0, 10, 0),
        decoration: BoxDecoration(
          color: Colors.blue,
          borderRadius: BorderRadius.circular(10),
          border: Border.all(color: Colors.black),
        ),
        alignment: Alignment.center,
        child: const Text('Hello World', style: TextStyle(color: Colors.white)),
      ),
    );
  }
}
```

Challenge:



Build this challenge before proceeding to the next page

Implement OOP

To reduce the number of lines in the code above and improve its structure, you can implement object-oriented programming (OOP) principles. By creating a separate class for the Container widget, you encapsulate its behavior and properties. Since this container widget doesn't change, it should be a stateless widget.

Not using OOP principles

main.dart

```dart
class MyHomePage extends StatelessWidget {
  const MyHomePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Counter Application'),
        backgroundColor: Colors.purple[200],
      ),
      body: Center(
        child: Column(
          children: [
            Container(
              height: 100,
              width: 100,
              margin: const EdgeInsets.fromLTRB(10, 0, 10, 0),
              decoration: BoxDecoration(
                color: Colors.red,
                borderRadius: BorderRadius.circular(10),
                border: Border.all(color: Colors.black),
              ),
              alignment: Alignment.center,
              child: Text('Hello World'),
            ),
            Container(
              height: 100,
              width: 100,
              margin: const EdgeInsets.fromLTRB(10, 0, 10, 0),
              decoration: BoxDecoration(
                color: Colors.blue,
                borderRadius: BorderRadius.circular(10),
                border: Border.all(color: Colors.black),
              ),
              alignment: Alignment.center,
              child: Text('NT'),
            ),
            Container(
              height: 100,
              width: 100,
              margin: const EdgeInsets.fromLTRB(10, 0, 10, 0),
              decoration: BoxDecoration(
                color: Colors.green,
                borderRadius: BorderRadius.circular(10),
                border: Border.all(color: Colors.black),
              ),
              alignment: Alignment.center,
              child: Text('BSU'),
            ),
          ],
        ),
      ),
    );
  }
}
```

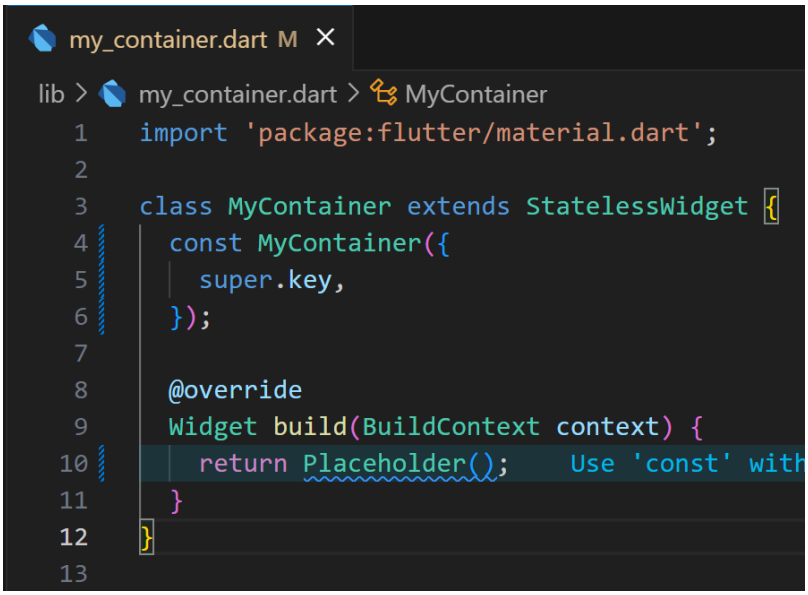
Using OOP Principle

main.dart

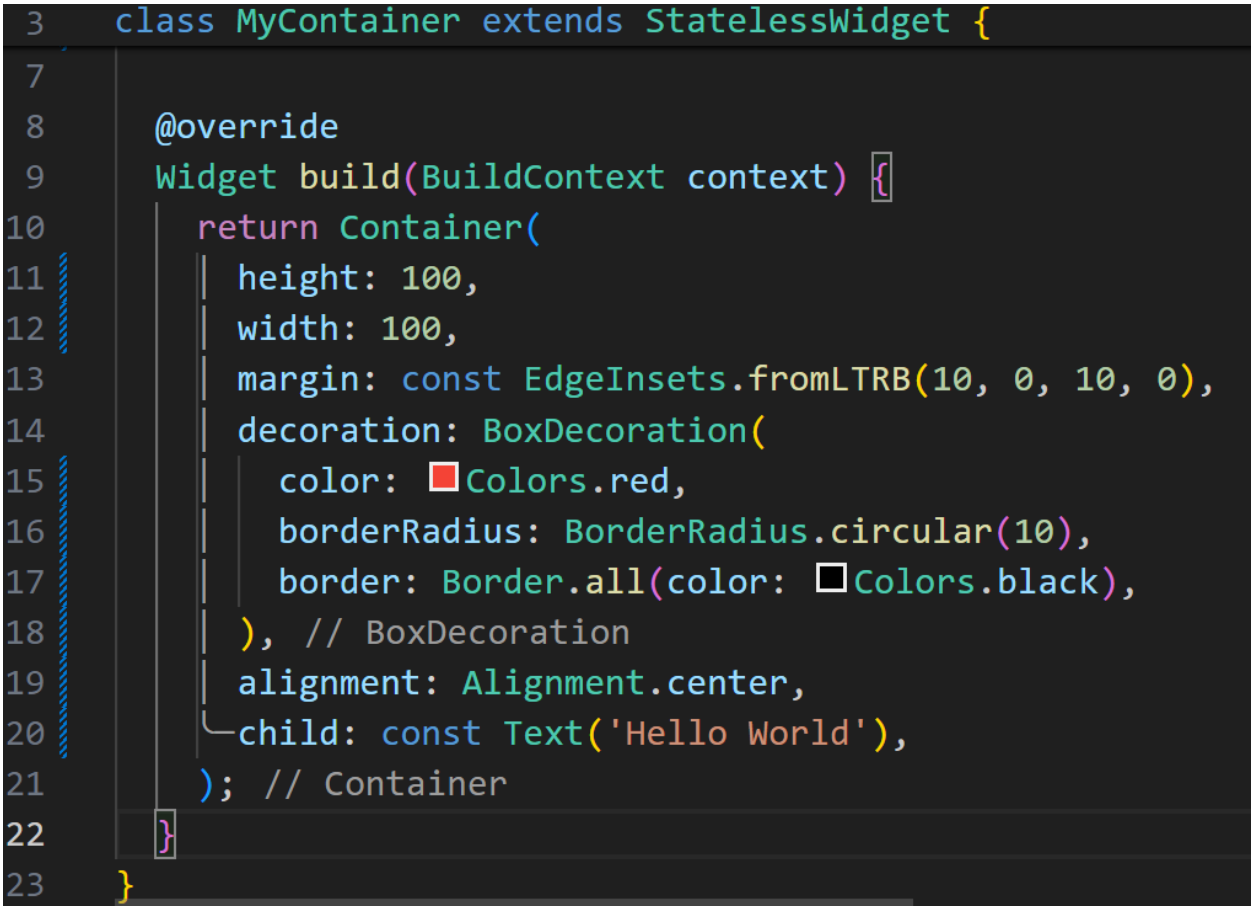
```dart
class MyHomePage extends StatelessWidget {
  const MyHomePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Counter Application'),
        backgroundColor: Colors.purple[200],
      ),
      body: const Center(
        child: Column(
          children: [
            MyContainer(
              text: 'Hello World',
              color: Colors.red,
            ),
            MyContainer(
              text: 'NT',
              color: Colors.blue,
            ),
            MyContainer(
              text: 'BSU',
              color: Colors.green,
            )
          ],
        ),
      ),
    );
  }
}
```

Steps on Implementing OOP Principles

Step 1: Create a new file or separate Class for the Container Widget



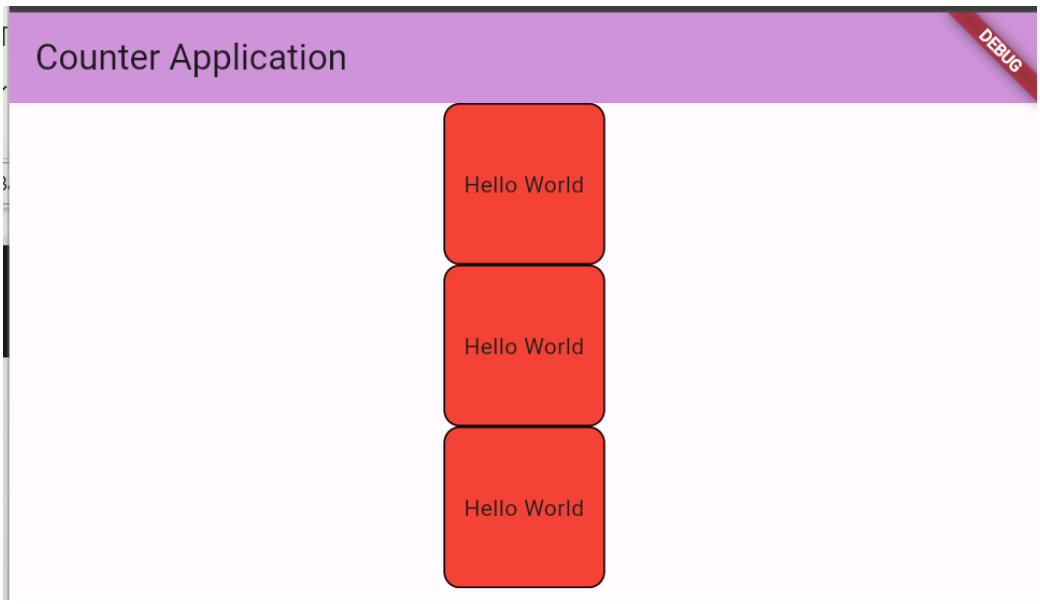
Step 2: Create the Container Widget



Step 3: Call the MyContainer on the homepage screen,

```dart
class MyHomePage extends StatelessWidget {
  const MyHomePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Counter Application'),
        backgroundColor: Colors.purple[200],
      ), // AppBar
      body: const Center(
        child: Column(
          children: [MyContainer(), MyContainer(), MyContainer()],
        ), // Column
      ), // Center
    ); // Scaffold
  }
}
```

Output



Three Red Containers with a hello world text should be displayed

Step 4: To understand which properties, need to be customized, we can pass parameters into the 'MyContainer' class. Specifically, the 'color' and text properties should be parameterized to allow for customization.

```
3   class MyContainer extends StatelessWidget {
4     const MyContainer({
5       super.key,
6       required this.text,
7       required this.color,
8     });
9
10    final String text;
11    final Color color;
```

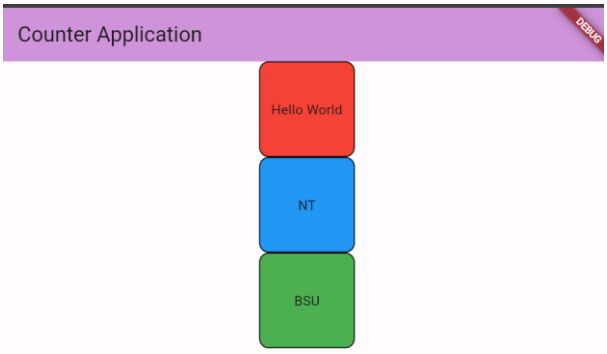
- Declare the datatype and variable name.
- Declare all variable names in the constructor. Marking them as required means that parameters must be provided for these variables.

Step 5: Use the variable names on the Container Widget

```
13    @override
14    Widget build(BuildContext context) {
15      return Container(
16        height: 100,
17        width: 100,
18        margin: const EdgeInsets.fromLTRB(10, 0, 10, 0),
19        decoration: BoxDecoration(
20          color: color, //Color
21          borderRadius: BorderRadius.circular(10),
22          border: Border.all(color: Colors.black),
23        ), // BoxDecoration
24        alignment: Alignment.center,
25        child: Text(text), //Text
26      ); // Container
27    }
28  }
```

Step 6: Add the parameters on the MyContainer() in the homepage

```
36          child: Column(
37            children: [
38              MyContainer(
39                text: 'Hello World',
40                color: Colors.red,
41              ), // MyContainer
42              MyContainer(
43                text: 'NT',
44                color: Colors.blue,
45              ), // MyContainer
46              MyContainer(
47                text: 'BSU',
48                color: Colors.green,
49              ) // MyContainer
50            ],
51          ), // Column
```



----------**DONE**----------

This way of implementing OOP can be applied to all widgets.

Optional passing of parameters

Declare different variables and provide its datatype. For optional variables, provide a default value on the constructor.

```
3   class MyContainer extends StatelessWidget {
4     const MyContainer({
5       super.key,
6       required this.text,
7       required this.color,
8       this.height = 100,
9     });
10
11    final String text;
12    final Color color;
13    final double height;
14
15    @override
16    Widget build(BuildContext context) {
17      return Container(
18        height: height, //Height
19        width: 100,
```

```
37            children: [
38              MyContainer(
39                text: 'Hello World',
40                color: Colors.red,
41                height: 200,
42              ), // MyContainer
43              MyContainer(
44                text: 'NT',
45                color: Colors.blue,
46              ), // MyContainer
47              MyContainer(
48                text: 'BSU',
49                color: Colors.green,
50              ) // MyContainer
51            ],
52          ), // Column
```

main.dart

Text Widget

The Text widget in Flutter is used to display a string of text with single style. It's a fundamental widget in Flutter, often used to show labels, descriptions, or any form of textual content in the app.

Text Widget Overview

- Purpose: To display a string of text with styling.
- Common Use Cases:
- Displaying headings, labels, and paragraphs.
- Showing messages, notifications, or descriptions.
- Customizing text appearance with different styles, colors, and alignments.

Basic Structure

```
Text('Hello, World!')
```

Properties

The Text widget has several properties that allow for customization:

1. **data**:
    - **Type**: String
    - **Description**: The text to display.
2. **style**:
    - **Type**: TextStyle
    - **Description**: Customizes the appearance of the text, such as font size, color, weight, etc.
3. **textAlign**:
    - **Type**: TextAlign
    - **Description**: Aligns the text within its container. Common values include TextAlign.left, TextAlign.center, TextAlign.right, etc.
4. **maxLines**:
    - **Type**: int
    - **Description**: Specifies the maximum number of lines for the text to span.

5. **overflow**:
    - **Type**: TextOverflow
    - **Description**: Defines how the text should handle overflow. Values include TextOverflow.clip, TextOverflow.fade, TextOverflow.ellipsis, etc.
6. **softWrap**:
    - **Type**: bool
    - **Description**: Indicates whether the text should break at soft line breaks.
7. **textScaleFactor**:
    - **Type**: double
    - **Description**: Scales the text by this factor.
8. **textDirection**:
    - **Type**: TextDirection
    - **Description**: The directionality of the text. Values include TextDirection.ltr (left to right) and TextDirection.rtl (right to left).
9. **semanticsLabel**:
    - **Type**: String
    - **Description**: Provides a semantic label for accessibility.

Example with Properties

```dart
Text(
  'Hello, Flutter!',
  style: TextStyle(
    fontSize: 24,
    fontWeight: FontWeight.bold,
    color: Colors.blue,
  ),
  textAlign: TextAlign.center,
  maxLines: 2,
  overflow: TextOverflow.ellipsis,
)
```